

# $\Omega$ -CWY: finite syntax and expansion rules

DEFINITIONS & PROOFS / ENGLISH EDITION

Version: 2026-09-17, using  $D[b+1]$  for limit labels.

This is a **candidate system**. Interpreting all formal labels as ordinals and proving well-ordering of the domain generated from the external top remain open tasks. The present release packages existing rules and local arguments, not a new global proof.

## 1. Finite, closed self-indexed structure

A finite term  $G$  is a finite column sequence  $C_0, \dots, C_j$ . The empty sequence is zero; column addresses start at zero. A row of column  $j$  is  $(p, P)$  with  $p < j$ , where  $P$  is a profile

$$P = (s ; (D_1, v_1), \dots, (D_t, v_t))$$

Its constraints are:

- $D_1 > \dots > D_t > 0$ . Each label is a **finite closed term of this same syntax**, never the external top.
- $s < v_1 < \dots < v_t$ .
- $s \leq p$ , and each  $v_i$  is either at most  $p$  or equal to the current column  $j$ .
- A constant profile stores only  $s$ .

Each nested label has its own local addresses. Outer relocation never enters that label. The syntax is a finite tree, not cyclic self-reference.

Regard  $P$  formally as a step function: it is  $s$  at and above  $D_1$ , changing to  $v_i$  when moving down across  $D_i$ . Its value at zero,  $q$ , is the *origin address*: the last  $v_i$  if there is a cut, and otherwise  $s$ . This description uses the recursive term order, without assuming that the entire order is well-founded.

Within a column parents strictly decrease and profiles strictly increase. This is the skyline normal form.

## 2. Comparison and skyline

Define comparison recursively on finite syntax:

1. Terms compare lexicographically by columns, with proper prefixes smaller.
2. Columns compare lexicographically by rows; a row compares by natural-number parent, then profile.
3. Profiles compare first by  $s$ , then lexicographically by their cut tables. A cut compares first by its label  $D$ , then value  $v$ . Proper table prefixes are smaller.

Label comparison recurses into strict subterms, so the comparison algorithm terminates. This is not a proof that the resulting order is a well-order.

For a finite collection of rows, skyline keeps the greatest profile for each parent, scans parents from largest to smallest, and retains only profiles strictly exceeding the last retained one.

## 3. Canonical display

The general form is

$$[p : (s | \{D_1\} : v_1, \{D_2\} : v_2, \dots)]$$

Braces contain complete nested expressions, not opaque label names. Semicolons separate rows, [ ] is an empty column, and  $\emptyset$  is the empty expression. In actual canonical output every record is written; ellipses above only describe the schema.

If all labels are finite integers, display  $P$  as an ordinary nondecreasing address word: write  $s$ , then repeat  $v_i$  exactly  $D_i - D_{i+1}$  times, with  $D_{t+1} = 0$ . For example, base 0 and cuts  $(3, 1), (1, 2)$  display as  $(\emptyset, 1, 1, 2)$ . **No repetition powers are used.**

Conversely, remove redundant initial copies of the root letter and recover cuts from changes of value in the finite word. A constant word normalizes to one letter.

## 4. Lower a profile in block b

Block indices begin at zero. For origin  $q = P(0)$ , scan the origin column  $C_q$  for the predecessor approximation to  $P$ :

- At each profile  $Q < P$ , remember  $Q$  and continue.
- Stop at the first row  $(r, Q)$  with  $Q \geq P$ . Here  $P$  has a least positive cut  $D$  and  $s \leq r < q$ .
- If every row is smaller, return the last profile, or the absent value if the column is empty.

At a stopping row define

$$\delta = \begin{cases} D[0], & D \text{ ends in an empty column,} \\ D[b + 1], & D \text{ ends in a nonempty column.} \end{cases}$$

Clip the old values to  $r$  above the new cut, keeping  $q$  below it:

$$L(\xi) = \begin{cases} q, & \xi < \delta, \\ \min(P(\xi), r), & \xi \geq \delta. \end{cases}$$

On finite tables, replace address values by their minimum with  $r$ , remove adjacent equal values, and append  $(\delta, q)$  if  $\delta$  is nonzero. The former origin plateau disappears or drops to  $r$ . Return the larger of  $L$  and the remembered smaller profile; if none was remembered, return  $L$ . This query does not itself graft a parent tail.

## 5. Local lowering of the last column

For nonempty last column  $j$ , let its final row be  $(c, P)$  and  $q$  its origin. Define  $D_b(G)$ :

1. Keep every last-column row except the final one.
2. Compute the origin predecessor approximation above. If present, add it with parent  $c$  and replace its address  $q$  by  $j$ .
3. Add all rows of parent column  $C_c$ , replacing  $c$  by  $j$  in their profile values.
4. Take skyline.

The lifts act only on natural-number profile values. They do not modify any nested label's internal addresses.

## 6. Fundamental sequences

Zero $[n]$  is zero. For any nonzero finite  $G$ ,  $G[0]$  deletes its last column. An empty last column marks a successor; all its indexed terms are this same deletion.

For a nonempty final column, let  $c$  be its last controlling parent,  $j$  its position and  $w = j - c$ . Retain  $C_0, \dots, C_{j-1}$  and for  $b = 0, \dots, n - 1$  append

$$\text{shift}_b(D_b(G)), \quad \text{shift}_{b+1}(C_{c+1}), \dots, \text{shift}_{b+1}(C_{j-1}).$$

The map  $\text{shift}_h$  fixes outer addresses below  $c$  and adds  $hw$  to addresses at least  $c$ . Both parents and profile values move; nested labels do not.

Each block has  $w$  columns and depends on  $b$ , not on the final  $n$ . Thus  $G[n]$  is a whole-column prefix of  $G[n+1]$ , strictly so for limits; successor terms coincide. Earlier columns remain unchanged.

One-step expansion only recursively calls expansion on strict input label subterms, and produces finitely many blocks. **One-step algorithm termination** follows by finite structural recursion, without assuming global well-ordering. Runtime protections are separate.

The local lowering/skyline mechanism is intended to produce a smaller first replacement column. It is not a substitute for complete proofs of closure of the designated domain, cofinality or well-ordering. Prefix preservation alone does not settle those questions.

## 7. External top and standard domain

For nonzero  $D$ , let  $S(D)$  be the two-column seed: an empty column followed by the sole row  $(\emptyset, (\emptyset; (D, 1)))$ . Set

$$T_0 = 1, \quad T_{n+1} = S(T_n), \quad \text{Limit}[n] = T_n.$$

Every application of  $S$  adds exactly one level of label nesting. The external top is just a generator for these finite candidates. It is not permitted inside a label, and no assertion is made that it enumerates all structurally legal handwritten terms. Its terms need not be prefixes of one another.

The studied standard domain is the finite reachability closure of these seeds under fundamental-sequence and predecessor operations. The parser checks local syntax, address bounds and normal form, not reachability.

The convention  $D[b+1]$  must not silently revert to  $D[b]$ . The latter traps every fundamental-sequence term of  $T_3$  below  $T_2$ . The chosen convention has passed finite adjacent-seed checks; that is not a general cofinality proof.

## 8. Counts and diagrams

For each prefix, hold earlier columns fixed and repeatedly apply  $D_0$  to its final column until that column is empty; then count its deletion once. This defines the count view. It is not the brute-force procedure of repeatedly taking the whole term's first fundamental-sequence entry.

A limit label inside  $D_0$  still uses its first indexed term under this version's rule. If time, local-step or kernel budgets are exhausted, preserve the already completed exact count prefix, and display  $?$  for the current and every subsequent column. Do not expose an unfinished partial count as an exact value.

For a small recognizable fragment, let  $B_k$  be one block consisting of a root empty column at  $a$  followed by  $k$  columns  $[a; (a)]$ . Interpret decreasing-exponent concatenations as

$$B_{k_1} \cdots B_{k_m} \longmapsto \omega^{k_1} + \cdots + \omega^{k_m}.$$

Their column order agrees with polynomial order. For final exponent  $k > 0$ , the zeroth term replaces  $B_k$  by  $B_{k-1}$ , and index  $n$  replaces it by  $n + 1$  copies of  $B_{k-1}$ . For  $k = 0$ , delete  $B_0$ . The fragment is closed under expansion.

$\omega\omega = [][\emptyset; (\emptyset)][1; (\emptyset)]$  has  $n$ th term  $B_{n+1}$ , the fragment's  $\omega^\omega$  bound. The current renderer recognizes the larger Cantor-tree fragment in the next section and does not impose a  $\omega^\omega$  drawing cutoff.

The diagram uses one track per parent relation. At level zero it marks the origin value, and at each cut it marks the values at and above the change. Sloping edges lead to parent-column base points. Every actual cut is shown; constant intervals do not duplicate points. Multiple parent rows use separate tracks within a column. A separate footer displays the full count sequence, including question marks; red addressing numbers are not replaced by counts.

## 9. Familiar labels below $\varepsilon_0$

Encode a finite Cantor normal form

$$\alpha = \omega^{\beta_1} + \cdots + \omega^{\beta_m}, \quad \beta_1 \geq \cdots \geq \beta_m,$$

as an ordered forest. A root represents one monomial, its children represent the monomials of its exponent recursively, and a leaf represents 1. Output nodes in preorder:

- An outer tree root at position  $a$  is an empty column.
- Each non-root node is a single row  $(p, (a))$ , with  $p$  its tree-parent address and the constant profile pointing to that **outer tree's root  $a$** .
- Each child list and the outer root list must be nonincreasing in represented monomial order.

This explicitly encodes all finite Cantor normal forms below  $\varepsilon_0$ . The display recognizer checks the single-row constant-root condition, preorder parent relation and decreasing ordering at every level. If any condition fails, it retains the raw expression. It does not assign an ordinal by arbitrarily normalizing a noncanonical forest.

|                                            |                                    |
|--------------------------------------------|------------------------------------|
| <code>[] [0: (0)]</code>                   | <code>-&gt; omega</code>           |
| <code>[] [0: (0)] [0: (0)]</code>          | <code>-&gt; omega^2</code>         |
| <code>[] [0: (0)] [1: (0)]</code>          | <code>-&gt; omega^omega</code>     |
| <code>[] [0: (0)] [1: (0)] [0: (0)]</code> | <code>-&gt; omega^(omega+1)</code> |

This recognition is independent of observed counts. The first differing parent pointer or root separator corresponds to the first differing Cantor exponent; the nonincreasing conditions make this the actual canonical comparison.

Expansion stays in the fragment. Write  $\alpha = \gamma + \omega^\beta$  for its last monomial:

- If  $\beta = 0$ , delete the empty last root, obtaining  $\gamma$ .
- If  $\beta$  is a successor, the last leaf's parent is the outer root. The lowered column is empty; the copied block is the tree for  $\omega^{\beta-1}$ . Thus  $\alpha[n] = \gamma + \omega^{\beta-1}(n+1)$ .
- If  $\beta$  is a nonzero limit, the last leaf's parent is not the outer root. Parent grafting and block copying occur inside the exponent tree; recursion gives  $\alpha[n] = \gamma + \omega^{\beta[n]}$ .

For example,  $\omega^{\omega+1}$  expands to  $\omega^\omega(n+1)$ . The displayed interpretation follows the forest rules, not just visual resemblance. It does not establish general  $\Omega$ -CWY well-ordering.

All finite labels are sorted by the notation's own comparator. Successfully recognized Cantor labels use ordinary powers, sums and coefficients. Larger or unrecognized labels retain their complete raw expression. Long labels wrap with increased vertical spacing, not truncation; explicit diagram/text/time budgets remain.

## 10. Conclusions not established

Neither this implementation nor its finite tests proves:

- absence of infinite descending chains in the standard domain;

- cofinality of every candidate fundamental sequence in that whole domain;
- a global embedding of  $wY$  or complete equivalence with CWY2 or the earlier omega-plus-one system;
- well-ordering provability in a specified axiom system;
- the intended ordinal meaning of every structurally legal but unreachable handwritten term.

Finite agreement with old CWY2 and computability of the self-indexed rules do not replace those missing conclusions. No new global proof is attempted for this release.

## 11. NER implementation and budgets

The standalone NER expander is named  **$\Omega$ -CWY**. Its internal ID `omega-cwy2-self-v1` remains for compatibility; the display name is not  $\Omega$ -CWY2. There are exactly three views: original expression, count sequence and mountain. Inputs include  $T0$ ,  $T2[3]$ ,  $Top[3]$ ,  $w$ ,  $w2$ ,  $ww$  and complete lists.

Readable sources are `core.mjs`, `counts.mjs`, `views.mjs` and `entry.mjs`. Run `node build.mjs` here to rebuild the same standalone file without third-party Node packages.

The core interface has roughly a 700ms budget; counting has about 450ms and 30,000 local lowering steps. Additional limits cover width, depth, structural size, text and caches. Counts use `BigInt`, so a resource limit does not mean a fixed integer bit width has been exceeded. Oversize diagrams report a complete failure rather than drawing a misleading fragment; count failures retain exact prefixes as described above.

Existing prefix/structural-recursion arguments and the Cantor-fragment explanation are included. There is no Lean certificate and no added global well-ordering, cofinality or  $wY$ -comparison theorem. Bounded implementation checks are reported in `VALIDATION`.